

Analysis: Bathroom Stalls

Test set 1

For test set 1, the limits are small enough that you can just simulate the rules outlined in the statement. Most implementations of a simulation will run in $O(NK)$ time and thus finish immediately, but even a slow $O(N^2K)$ implementation like "try every possible stall for the next person, and for each empty stall run a loop for each side to check for the closest neighbors" will most likely finish in time.

For test sets 2 and 3, however, something quadratic in the number of stalls won't cut it, so we have to do better.

Test set 2

The critical observation to jump from test set 1 to test set 2 is that only the number of consecutive runs of empty stalls matters at any given time. The next person always chooses the middle stall or the left of the two middle stalls of a longest subsequence of consecutive empty stalls. Moreover, the output format already hints at this: even if you were to choose the rightmost of a set of two middle stalls, or a longest run of stalls other than the leftmost one, the answer would not change. Thus, we can rewrite the algorithm in this equivalent (for the required output) form:

1. Find any longest subsequence of consecutive empty stalls.
2. Choose the middle or one of the two middle stalls.

Notice that even though there are still ties to be broken, the output is equivalent for all of them. Since the output is equivalent, so is the multiset of lengths of consecutive runs of empty stalls left behind, so the whole process only depends on that multiset. (As a reminder, a multiset is a set in which the same element can appear more than once.) We can write an optimized simulation that solves test set 2 following this pseudocode:

```
S = {N} - This is a multiset!
repeat K times:
    X = max(S)
    X0 = ceil((X - 1) / 2)
    X1 = floor((X - 1) / 2)
    if this is the last step:
        we are done; answer is X0 and X1
    else:
        remove one instance of X from S
        insert X0 and X1 into S
```

If the operations over S are efficient, this will run in quasilinear time. There are many data structures that support insertion, finding the maximum, and removal of the maximum in logarithmic time, including AVL trees, red-black trees, and heaps. Many languages have one such structure in their standard libraries (e.g., the `multiset` or `priority_queue` in C++, `TreeSet` in Java, and `heapq` module in Python). Since we take $O(\log K)$ time for each of K steps, the algorithm takes only $O(K \log K)$ time, which is fast enough to solve test set 2. However, for test set 3, even quasilinear time on K is not enough.

Test set 3

The observation required to solve test set 3 is that we are simulating similar steps over and over again. The first time a bathroom user arrives, we partition N into $\text{ceil}((N - 1) / 2)$ and $\text{floor}((N - 1) / 2)$, which means that numbers between $\text{ceil}((N - 1) / 2)$ and N will never appear in S . This hints at a logarithmic number of simulation steps.

Let's divide the work in stages. The first stage processes only N . Then, stage $i+1$ processes all of the values spawned by stage i . So, stage 2 processes up to 2 values: $\text{ceil}((i - 1) / 2)$ and $\text{floor}((i - 1) / 2)$. What about the other stages? It is not hard to prove by induction that they also process at most two consecutive values: since stage i processes two consecutive values, they are either $2x$ and $2x+1$ or $2x$ and $2x-1$, for some x (that is, one even and one odd number). Thus, the spawned values for stage $i+1$ can only be x and/or $x-1$. Since the largest value in each stage is at most half the largest value of the previous stage, there are a logarithmic number of stages. This all means that there are at most $O(\log N)$ different values that go into S at any point. Of course, some of them appear in S many, many times. So, the optimization to get the running time low enough for test set 3 is to process all repetitions of a given value at the same time, since all of them yield the same X_0 and X_1 values. We can do that by using a regular set with a separate count for the number of repetitions.

```
S = {N} - This is a set, not a multiset!
C(N) = 1
P = 0
repeat:
    X = max(S)
    X0 = ceil((X - 1) / 2)
    X1 = floor((X - 1) / 2)
    P = P + C(X)
    if P ≥ K:
        we are done; the answer is X0 and X1.
    else:
        remove X from S
        insert X0 and X1 into S
        add C(X) to the counts of X0 and X1 in C
```

Once again, we have structures that implement all the required operations in logarithmic time, yielding an $O(\log^2 N)$ running time overall. In general, adding any good dictionary implementation to the structure of choice from the test set 2 solution would work, either by plugging the dictionary functionality into the structure (like `map` in C++ or `TreeMap` in Java) or having a separate hash-table for the dictionary (which is the easiest implementation in Python).

Moreover, since we proved the population of S is at most 4 at any given time (only values from two consecutive stages can coexist in S), any implementation of set and dictionary will provide all operations in constant time, because the size of the whole structure is bounded by a constant! This makes the overall time complexity just $O(\log N)$.

This was a nice problem to put experimentation to work if your intuition was not enough. After solving test set 1, if you print the succession of values for a fixed N , you may spot the pattern of few values occurring in the set S , and from there, you can find the mathematical arguments to support the needed generalization. In harder problems in later rounds, this can become an even more important asset to tackle problems. As you can see in many parts of last year's finals [live stream](#), finalists use experimentation a lot to inspire themselves and/or validate their ideas before committing to them.

Analysis: Fashion Show

Fashion Show: Analysis

What's really going on here?

The somewhat strange scenario presented in this problem disguises a classic type of chess problem in which pieces must be placed so that they do not attack each other. The [eight queens puzzle](#) is one well-known example. Our problem is more complicated, and it involves three types of piece. Let's restate the rules about how the models can interact in more chess-like terms:

- The + models are bishops. Two bishops may not occupy the same diagonal.
- The x models are rooks. Two rooks may not occupy the same row or column.
- The o pieces are queens. Two queens may not occupy the same row, column, or diagonal. Moreover, a queen and a bishop may not occupy the same diagonal; a queen and a rook may not occupy the same row or column.

Observe that our problem does not accord with typical chess rules about "attacks". For example, in our problem, it is fine for a rook and a bishop to share the same row or column. Also, unlike in chess, a piece between two pieces does not prevent them from "attacking" each other. For example, we do not allow two bishops to share the same diagonal even if there is a rook between them.

Decomposing the problem

How will we deal with this variety of pieces? The critical insight is that the rook and bishop parts of the problem are independent; we can solve them separately, placing as many new pieces in each subproblem as possible, and then merge the answers together. A queen is just a rook plus a bishop; we can add each pre-placed queen into the rook subproblems as a rook and into the bishop subproblem as a bishop. Then, once the subproblems are solved, we can turn any cell that is occupied in both subproblem solutions back into a queen.

This strategy is guaranteed not to violate any rules. A rook subproblem solution will never have two rooks in the same row or column, and a bishop subproblem solution will never have two bishops on the same diagonal. Merging the two subproblems' solutions may generate new queens, but it is impossible for them to violate any rules, since that would imply a rule violation in one of the subproblems. For example, we do not need to worry that we will end up with a queen and a bishop on the same diagonal, since that would only be possible if our bishop solution had two bishops on the same diagonal.

Moreover, as long as we place as many rooks as possible in the rook subproblem, and as many bishops as possible in the bishop subproblem, we are guaranteed the maximum possible amount of style points for our test case. Since queens are worth 2 points, merging a rook and a bishop into a queen has no effect on our score.

Let's walk through an example. This case:

```
+ . .  
+ . o  
x . .
```

can be decomposed into rook and bishop problems:

```

... +..
..x +.+
x.. ...

```

which can be solved independently:

```

.x. +..
..x +.+
x.. +..

```

and then merged back together. Note that we have replaced the former $\times$ model from the lower left corner with an $\circ$ model.

```

+x.
+.o
o..

```

Solving the subproblems

Now all we need are strategies for the subproblems themselves. The rook problem is straightforward. Each rook removes exactly one row and column from further consideration, so any greedy placement strategy that does not violate the rules will place exactly N rooks.

The bishop subproblem is more challenging. We can approach it differently in the Small and Large datasets.

Bishops: Small dataset

In the Small dataset, any pre-placed pieces are all in the top row. Observe that bishops in the same row or column cannot possibly "threaten" each other, and so we can safely add a bishop to any top row cell that does not already have one. So, if we can come up with a general solution pattern in which the top row is always completely filled with bishops, then we can solve any Small test case, because we can safely turn any pre-placed arrangement into a row packed with bishops.

Once the top row is filled with bishops, where else on the board should we put them? The bottom row is farthest away from the constraints imposed by the top row, and we can try putting a bishop in every bottom-row cell except for the cells on either end (which are "threatened" by the bishops at the two ends of the top row). These bishops do not threaten each other or any top row bishops, so this arrangement is valid, and we have a total of $2N-2$ bishops. No additional bishops can be added after that, though. Have we really placed as many as possible?

At this point, we can experiment and convince ourselves that this solution is probably optimal. We can also take a leap of faith; for a Small dataset in a Qualification Round, there is little incentive not to submit the solution and see if it is correct. Or, we can come up with a proof. An N by N board has $4N-2$ different diagonals. Moreover, the parallel diagonals of length 1 in opposite corner cells can never both be used simultaneously, so there are really only $4N-4$ simultaneously usable diagonals. Since placing a bishop uses up two diagonals, $2N-2$ is an upper bound on the number of bishops we can place. So, our method is optimal!

We must still take care, though, to handle a pre-placed rook/queen correctly if one is present, and to merge the rook and bishop solutions appropriately, creating queens when necessary. We must also be careful with the 1 by 1 board, which has no bottom row distinct from its top row.

It is possible to come up with the same construction without realizing that the problem can be decomposed into rooks and bishops; it is just more difficult to justify the optimality of the construction in that case!

Bishops: Large dataset

One helpful observation is that, just as in a chess game, the "white cell" bishops (in our problem, cells for which $R_i + C_i$ is even) are completely independent of the "black cell" bishops (cells for which that sum is odd). So we can consider these as sub-sub-problems.

You can use a [bipartite matching algorithm](#) to place the bishops optimally. There is, however, a greedy strategy; unlike in the rook subproblem, however, not just any greedy strategy will work!

Let's consider an 8 x 8 board with no pre-placed bishops. We'll look at just the "black" cells of the board, and tilt the board 45 degrees clockwise. (Here, .s represent black cells. The @s do not represent cells — they are just there to orient the image.)

```
@@@..@@@
@@....@@
@.....@
.....
@.....@
@@....@@
@@@..@@@
```

This new board has an important property: any row is a subset of all rows with more black cells than it. For example, the four black cells in the second row are also present in every row with at least four black cells. This property holds regardless of the value of N , or whether we look at "white" or "black" cells. It even holds as we add bishops! Adding a bishop wipes out one entire row and one entire column — notice that we have made this more like the rook problem — and since the remaining rows have all lost the same column, the aforementioned property is unchanged.

The property suggests a greedy strategy: first, sort the rows by the number of available cells. Then, pick a "smallest" row (one with a minimal number of available cells), place a bishop in any column in that row, and wipe out that row and column. This strategy is guaranteed to place an optimally large number of bishops. Suppose that we choose a column C in the smallest row R , and another column C' in some other row R' , such that C' is also in R . Then C must also be in R' , since all columns in the smallest row are in every other row. We can therefore swap them over, and it is equally valid to choose C' in row R and C in row R' .

One tempting (but incorrect) greedy strategy is to go left to right, top to bottom, and greedily place bishops in all legal places. Here is an example of a board for which that fails.

```
.@@@
....@
...@@
..@@@
.@@@
```

The incorrect greedy strategy will only place three bishops, as follows:

```
+@@@
.+..@
..+@@
..@@@
.@@@
```

whereas the correct strategy will place four (here is one optimal placement):

. @ @ @ @
. . . + @
. . + @ @
. + @ @ @
+ @ @ @ @

Notice that although we can always place **N** rooks, the number of bishops we are able to place depends on the pre-placed bishops. This explains why different test cases with the same value of **N** might have different maximum scores.

Analysis: Oversized Pancake Flipper

Oversized Pancake Flipper: Analysis

Let's start with two simple but key observations. First, the flips are commutative. That is, the order in which you make the flips doesn't matter: flipping at the same positions in any order always yields the same result. Second, you never need to flip at the same position more than once: since flips are commutative, you may rearrange the flips so that two flips at the same position happen consecutively, and then it is clear that they cancel each other out and you might as well not do either.

Small dataset

Applying the observations above to the Small dataset leaves only a really small number of combinations to try. Since there are $N-K+1$ possible flips, the number of flip subsets is 2^{N-K+1} , which is at most $2^9 = 512$ for this dataset. We can just try all of these combinations, discard the ones that leave at least one pancake blank side up, and get the minimum number of flips or determine that it is impossible.

Even if you don't have the observations above, a [breadth-first search](#) of all possible pancake states, using all possible flips as transitions, is also easily fast enough.

Large dataset

Of course, the approach above will take way too long for the Large dataset, which can have up to 1000 pancakes. There is a simple greedy strategy that is not too hard to find. Notice that the left-most pancake p_1 is only affected by the left-most flip f_1 . That means that the initial status of p_1 completely determines what we need to do with f_1 if we want to have any chance of leaving all pancakes happy side up: use f_1 if and only if p_1 is initially blank side up. After deciding on f_1 , we can notice that there is only one remaining flip f_2 that affects the second to the left pancake p_2 . So, after we have used f_1 (or not), the current state of p_2 completely determines whether to use f_2 .

Continuing with this reasoning, we can use the leftmost $N-K+1$ pancakes to determine all flips. After doing that, we only need to check whether the last $K-1$ pancakes are happy side up or not. If they all are, then the answer is the number of flips we used. Otherwise, the answer is IMPOSSIBLE. Notice that this reasoning also implies that at most one of the subsets of flips from our Small-only solution above would have worked.

If we carry out the above strategy literally and actually flip all of the pancakes in memory each time, the complexity is $O(N^2)$. We can reduce this to $O(N)$ by only flipping one pancake at a time, and keeping a running count of the number of flips that we "owe" the current pancake. For instance, suppose that $N = 10$ and $K = 5$, and the first pancake is blank side up. We start with the first pancake, and because we must flip it, we know we must flip the first five pancakes. We increase our flip count to 1, and also make a memo to decrease the flip count by 1 once we reach the sixth pancake. We continue from left to right, and whenever a pancake is blank side up and the flip count is even, or a pancake is happy side up and the flip count is odd, we increase the flip count and make another memo. These memos can be easily stored in another array of length N that is checked before handling each pancake. This optimization is not necessary to solve the Large dataset, but it is a useful trick in programming contest problems

(and at least one of your Code Jam engineers has used it in day-to-day software engineering work!)

Analysis: Tidy Numbers

Tidy Numbers: Analysis

Small dataset

The Small dataset can be solved by a simulation, going backwards through the numbers that Tatiana counted. We start with N and check whether it is tidy. If it is, we are done; if not, we check $N-1$, and then $N-2$ if $N-1$ is not tidy, and so on.

Checking a number for tidiness takes a single pass over its digits. We can bound the number of digits of numbers no greater than N by $\log_{10} N$. Since we iterate through at most $O(N)$ numbers, the overall time complexity of this approach is $O(N \log N)$. This is easily fast enough to solve the Small dataset, in which $N \leq 1000$, but it will not hold up for the Large dataset, which can have N as large as 10^{18} . If you start at the non-tidy number 11111111111111110, you have a very long way down to count before reaching the answer, 9999999999999999!

To iterate through the digits of an integer, you can use a language-provided utility to convert it to a string, or repeatedly take the value modulo 10 to obtain the last digit and divide by 10 to remove the last digit.

Large dataset: A greedy approach

There is an efficient greedy approach for the Large, but it has some tricky cases.

Let us name the digits of N $N_1, N_2, \dots, N_L$ from most to least significant (i.e., left to right, the way in which we usually write digits). Let A be our desired answer: the most recently counted tidy number. We want A to have as many digits from the left as possible in common with N , because that minimizes the value $N - A$.

Find the first "inversion" in N — that is, the minimum i such that $N_i > N_{i+1}$. If there is no such i , then N is already tidy and $A = N$. Otherwise: no number starting with the sequence $N_1, N_2, \dots, N_i$ can be both smaller than N and tidy. So, we can try making our answer start with $A_1 = N_1, A_2 = N_2, \dots, A_{i-1} = N_{i-1}$. The next digit A_i has to be less than N_i , so we can try using $A_i = N_i - 1$. If A_i is greater than or equal to A_{i-1} , we can make the remaining digits from A_{i+1} onward into 9s; this makes A as large as possible while ensuring that it remains tidy. However, if A_i is less than A_{i-1} , we would just be creating another inversion, and so we should instead try starting A with the digits up to N_{i-2} . If that doesn't work, we can try the digits up to N_{i-3} , and so on. We may even start with zero digits of N ; for $N = 211$, we get $A = 199$. Even starting with zero digits might not work: for $N = 100$, for example, the answer is 99.

Let's condense these observations into a final algorithm. If there is no inversion, we are done. Otherwise, if the first digit of the inversion is i , find the maximum $j < i$ such that $N_j < N_{j+1}$. If there is no such j , set $j = 0$. Then A starts with $N_1, N_2, \dots, N_j, N_{j+1} - 1$, and then ends with enough 9s to make it length L . The only exception is if $j = 0$ and N_1 is 1, in which case the answer is $L-1$ 9s.

Since this strategy requires only one pass forward through the digits of N and then another pass backward, it is $O(\text{the number of digits in } N)$, which, as we argued above, is $O(\log N)$. In this case, we can even get away with not converting the input data strings into integers!

To bypass some of the complexity above, we could have simply tried all combinations of the form $SD999\dots99$, where S is each prefix of N (including the empty prefix), D is each possible digit from 0 to 9, and the number of 9s is such that the total length is L . $L-1$ 9s must also be included as a special case. Then the answer must be among those options, and it is the largest tidy number among them that does not exceed N . This is a common trick to simplify the code of greedy solutions: try more options than are needed, as long as the number of options is still tractable. It is often easier to implement finding the best solution among several options than to implement the checks necessary to avoid options we know we do not need.

Large dataset: A combinatoric approach

Another way to solve this with a little bit more math is to notice that there are few tidy numbers. For a fixed length L , the number of tidy numbers is the number of ways to put 8 balls in $L+1$ bins. Each bin represents a position at the start or end or in between two digits, and each ball represents "increase the number by 1". So, for instance, tidy number 2455 is represented by 1 ball in the first bin (skipping 1), 2 balls in the second bin (moving from 2 to 4), 1 ball in the third (4 to 5), no ball in the fourth (5 is repeated), and the rest of the balls in the last bin (moving from 5 up to 9, but there are no digits left to write). 8 balls in $L+1$ bins is equal to $(L+8 \text{ choose } 8)$ which, for the maximum $L=18$, is less than 2 million. So, we can enumerate all of the tidy numbers, skipping the rest and just return the largest we find which is no greater than N . To enumerate tidy numbers, we can use recursion like in this pseudocode:

```
best = 1
enum(current_string, current_digit, digits_left):
    if digits_left > 0
        enum(current_string + current_digit, current_digit, digits_left - 1)
        enum(current_string + (current_digit + 1), current_digit + 1, digits_left - 1)
    else
        if best ≤ string_to_int(current_string) ≤ N
            best = string_to_int(current_string)
```

We can also define a function that finds the next tidy number greedily and use it. We present such a function in the next subsection.

Large dataset: A binary search approach

Our original problem is: given an integer N , find the largest $Y \leq N$ such that Y is a tidy number. Let's consider a related problem: given an integer X , find the smallest integer $Y \geq X$ such that Y is a tidy number. That problem turns out to have a significantly simpler solution. Suppose that X is a sequence of digits $X_1, X_2, \dots, X_L$, enumerated from most to least significant, as above. Then Y can be formed by finding the first inversion in the sequence (i.e., the minimum i such that $X_i > X_{i+1}$) and creating a new number that is $X_1, X_2, \dots, X_i$ plus enough additional copies of X_i to make Y as long as X . For example, if we start with the number 13254, then the first inversion is at 32, and we replace everything after the 3 with more 3s, forming 13333. If there are no inversions, then X is already tidy and $Y = X$. This algorithm is $O(L)$ for an X of L digits.

With this related problem solved, we can solve the original problem by [binary searching](#) for the smallest range $[X, N]$ such that the Y (as defined above) that corresponds to X is not larger than N . This approach takes $O(\log^2 N)$ time, because the binary search takes $\log N$ steps, and each step requires running the $O(L)$ greedy algorithm above. As argued above, L can be bounded by $\log N$. This approach is less efficient than the greedy approach above, but it is still easily fast enough to pass the Large dataset.